1. Tracing the code there is a fault on line 3:
   for for (int i = 1 ; i < a t t e n d a n c e r e c o r d s . s i z e ( ) ; ++i)\
   The code starts on i = 1 skipping the first lecture. C++ vectors, which this is iterating through start at index 0. This means that the first lecture is effectively ignored for all cases and its respective attendance record is ignored for the purposes of pass/fail for the student.

2. It is impossible since the indexing error will always be hit every time the program is run, to fix this the error in the code would need to be fixed. There is also no way to call this function without running the for loop with the broken vector iteration loop. The error happens only for lecture one which is not properly counted. The fault is ALWAYS executed but the error case is not always hit.
3. The fault is executed but the error makes not difference on the outcome, since the index zero is always counted as attending if the user is already attending on the first lecture there is no error and the output would be correct:
   run("Test A (Q3): executes fault, no error state", {1,1,1,1,1,1,1,1,1,1});
   run("Test A2 (Q3): executes fault, no error state, absences elsewhere", {1,0,1,0,1,1,1,1,1,1});
4. Error test case with no failure, the following case produces the adequate 3+ absence case so it doesn't fail, however the code misses the first class so there should be 4 absences but the system only tracks 3.
   run("Test B (Q4): error state, no failure", {0,0,0,0,1,1,1,1,1,1});
5. Test cases that fail: The only true failure case happens when the student misses the first lecture and misses exactly 2 more, this results in a 2 / 3 output vs actual absences meaning the student should fail the course but does not because the system misses the first, all of the other cases the code encounters the error but it does not have an impact on the output that results in a fail except for the following 2:
   run("Test C (Q5): failure", {0,0,1,1,1,1,1,1,1,0});
   run("Test D (Q5): failure (minimal)", {0,0,0,1,1,1,1,1,1,1});
   Technically any combo of 0,x,x,x,x,x,x,x where exactly 2 other x's are 0s fit this criteria.

   Following is the code used to test if these cases return a failure, error or none.

```cpp
#include <iostream>
#include <vector>
#include <string>
using namespace std;


bool fail_lecture(const vector<int>& attendance_records, int* count_out = nullptr) {
    int absent_count = 0;
    for (size_t i = 1; i < attendance_records.size(); ++i) {   // FAULT: should be i = 0
        absent_count += attendance_records[i] == 0;
    }
    if (count_out) *count_out = absent_count;
    return absent_count >= 3;
}


bool fail_lecture_correct(const vector<int>& a, int* count_out = nullptr) {
    int absent_count = 0;
    for (size_t i = 0; i < a.size(); ++i) absent_count += a[i] == 0;
    if (count_out) *count_out = absent_count;
    return absent_count >= 3;
}

void run(const string& name, const vector<int>& rec) {
    int actual_cnt, expected_cnt;
    bool actual = fail_lecture(rec, &actual_cnt);
    bool expected = fail_lecture_correct(rec, &expected_cnt);
    cout << name << "\n  record: ";
    for (int x : rec) cout << x << ' ';
    cout << "\n  absent_count  expected=" << expected_cnt << " actual=" << actual_cnt
         << (expected_cnt == actual_cnt ? "  (state OK)" : "  (ERROR STATE)")
         << "\n  output        expected=" << boolalpha << expected << " actual=" << actual
         << (expected == actual ? "  (no failure)" : "  (FAILURE)") << "\n\n";
}

int main() {
    run("Test A (Q3): executes fault, no error state", {1,1,1,1,1,1,1,1,1,1});
    run("Test A2 (Q3): executes fault, no error state, absences elsewhere", {1,0,1,0,1,1,1,1,1,1});
    run("Test B (Q4): error state, no failure", {0,0,0,0,1,1,1,1,1,1});
    run("Test C (Q5): failure", {0,0,1,1,1,1,1,1,1,0});
    run("Test D (Q5): failure (minimal)", {0,0,0,1,1,1,1,1,1,1});
    return 0;
}
```